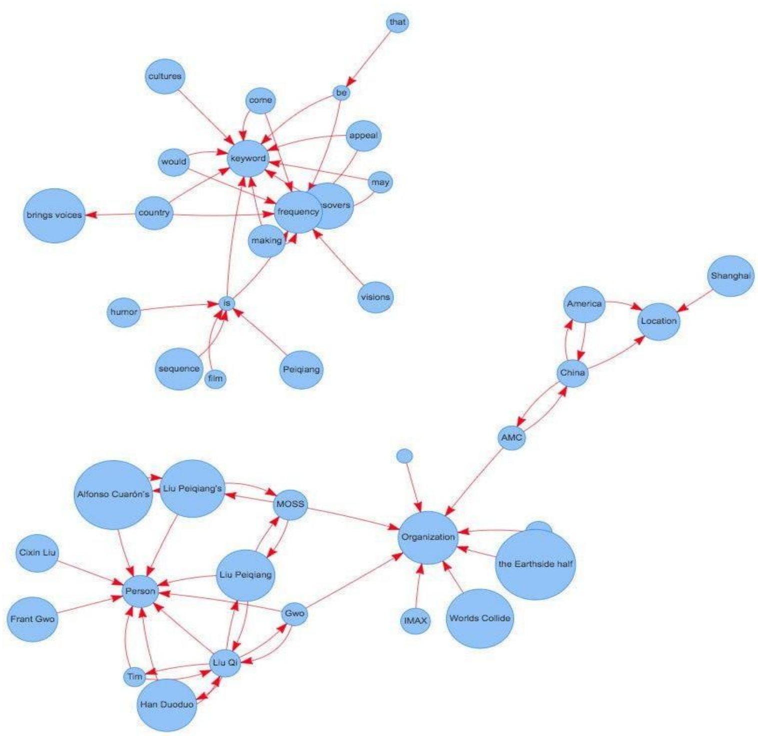
	Marwadi University **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** TextRank for Document Summarization	
Experiment No: 10	**Date:**	**Enrolment No:92301733043**

Aim: TextRank for Document Summarization

IDE: Google Colab

Theory:

TextRank is an algorithm based on PageRank, which often used in keyword extraction and text summarization. In this article, I will help you understand how TextRank works with a keyword extraction example and show the implementation by Python.

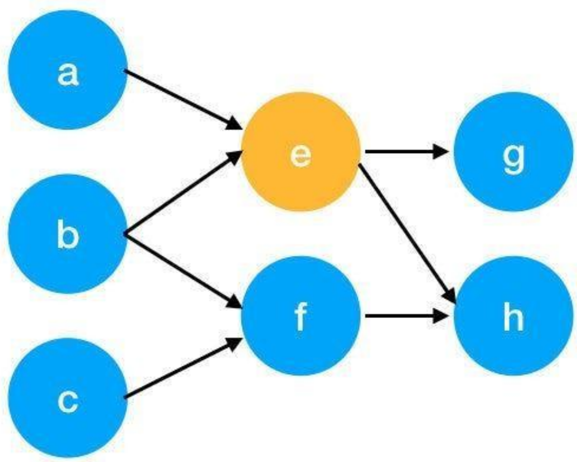


Understand PageRank

There are tons of articles talking about PageRank, so I just give a brief introduction to PageRank. This will help us understand TextRank later because it is based on PageRank. PageRank (PR) is an algorithm used to calculate the weight for web pages. We can take all web pages as a big directed graph. In this graph, a node is a webpage. If webpage A has the link to web page B, it can be represented as a directed edge from A to B. After we construct the whole graph, we can assign weights for web pages by the following formula.

$$S(V_i) = (1-d) + d * \sum_{j \in In(v_i)} \frac{1}{|Out(V_j)|} S(V_j)$$

- S(Vi) - the weight of webpage i
- d - damping factor, in case of no outgoing links
- In(Vi) - inbound links of i, which is a set
- Out(Vj) - outgoing links of j, which is a set
- |Out(Vj)| - the number of outbound links



Here is an example to better understand the notation above. We have a graph to represent how web pages link to each other. Each node represents a webpage, and the arrows represent edges. We want to get the weight of webpage e. We can rewrite the summation part in the above function to a simpler version.

$$In(v_e) = \{a, b\}, j \in \{a, b\}$$

$$\sum_{j \in \{a,b\}} \frac{1}{|Out(V_j)|} S(V_j) = \frac{1}{|Out(V_a)|} S(V_a) + \frac{1}{|Out(V_b)|} S(V_b)$$

$$= \frac{1}{|\{e\}|} S(V_a) + \frac{1}{|\{e, f\}|} S(V_b)$$

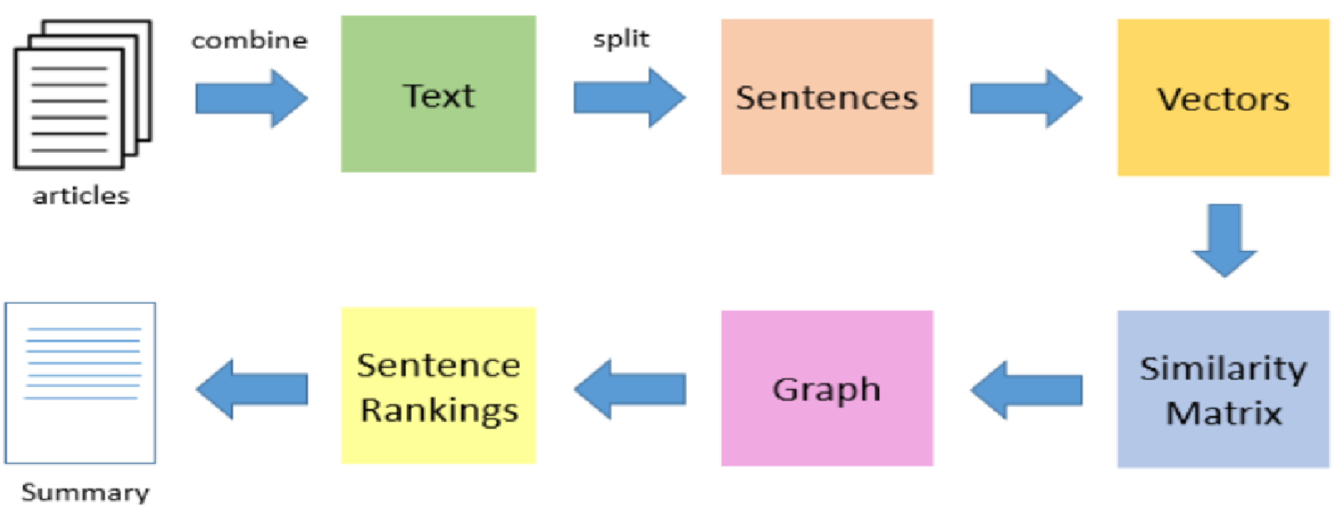
$$= S(V_a) + \frac{1}{2} S(V_b)$$

We can get the weight of webpage e by the following function.

$$S(V_e) = (1 - d) + d * \left(S(V_a) + \frac{1}{2} S(V_b) \right)$$

We can see the weight of the webpage e is dependent on the weights of its inbound pages. We need to run this iteration much time to get the final weight. In the initialization, the importance of each webpage is 1.

TextRank for document Summarization



TextRank works in the following steps:

1. Tokenize documents into sentences.
2. Preprocess each sentence in the document.
3. Count key phrases and normalize them or produce TFIDF Matrix, you can also use any kind of vectorization such as spacy vectors.
4. Calculate the Jaccard Similarity between sentences and key phrases.
5. Rank the sentences with higher significance.

Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	**Aim:** TextRank for Document Summarization	
Experiment No: 10	**Date:**	**Enrolment No:92301733043**

Pre Lab Exercise:

1. How keyword extraction is similar to document summarization process?

   Both **keyword extraction** and **document summarization** use the same underlying idea of **importance ranking**.

   In **TextRank**, both tasks:
   - Build a **graph structure** (words or sentences as nodes).
   - Use **similarity relationships** (edges).
   - Apply a **ranking algorithm (PageRank)**.

   Both aim to **identify the most important parts of the text**:
   - Keyword extraction → important **words/phrases**
   - Summarization → important **sentences**

2. How keyword extraction is different to document summarization process

Feature	Keyword Extraction	Document Summarization
Output	Words / phrases	Sentences / paragraphs
Goal	Identify key terms	Generate concise text
Nodes in graph	Words	Sentences
Meaning	No full context	Maintains context
Use case	Indexing, tagging	Reading, understanding

3. Limitation of TextRank as a summarizer

   Does not understand **deep meaning or context**

   Ignores **grammar flow between sentences**

   Important sentences may be missed if:
   - They don't have high similarity with others

   Works poorly on:
   - Very **short texts**
   - Highly **technical or complex documents**

   No **semantic understanding**, only statistical similarity

Program (Code):

```
#textrank algorrithm for document summarization
text = """
    India, officially the Republic of India,[j][20] is a country in South Asia. It is the seventh-largest country by area; the most populous country since 2023;[21] and, since its independence in 1947, the world's most populous democracy.[22][23][24] Bounded by the Indian Ocean on the south, the Arabian Sea on the southwest, and the Bay of Bengal on the southeast, it shares land borders with Pakistan to the west;[k] China, Nepal, and Bhutan to the north; and Bangladesh and Myanmar… nationalist movement emerged in India, the first in the non-European British Empire and an
```

influence on other nationalist movements.[54][55] Noted for nonviolent resistance after 1920,[56] it became the primary factor in ending British rule.[57] In 1947, the British Indian Empire was partitioned into two independent dominions, a Hindu-majority dominion of India and a Muslim-majority dominion of Pakistan. A large-scale loss of life and an unprecedented migration accompanied the partition.[58]"""

```
#importing the basic libraries
import numpy as np
import pandas as pd
import networkx as nx
import nltk
import re
from nltk.corpus import stopwords
from nltk.tokenize import sent_tokenize,word_tokenize

nltk.download('punkt')
nltk.download('stopwords')

#pre-processsing step
def preprocess_sentence(sentences):
  stop_words= set(stopwords.words('english'))
  words = word_tokenize(sentences.lower())
  words = [w for w in words if w.isalnum() and w not in stop_words]
  return words

def sentence_similarity(sent1,sent2):
  word1 = preprocess_sentence(sent1)
  word2 = preprocess_sentence(sent2)
  all_words = list(set(word1+word2))
  vector1=[word1.count(w) for w in all_words]
  vector2=[word2.count(w) for w in all_words]
  if sum(vector1) == 0 or sum(vector2) == 0:
    return 0
  #cosine similarity
  return np.dot(vector1,vector2)/(np.linalg.norm(vector1)*np.linalg.norm(vector2))

def build_similarity(sentences):
  n = len(sentences)
  sim_matrix=np.zeros((n,n))
  for i in range(n):
    for j in range(n):
      if i!=j:
        sim_matrix[i][j]=sentence_similarity(sentences[i],sentences[j])
  return sim_matrix

def textrank_summarization(text,topn=3):
  sentences=sent_tokenize(text)
  sim_matrix=build_similarity(sentences)
```

```
    graph=nx.from_numpy_array(sim_matrix)
    scores=nx.pagerank(graph)
    ranked_sentences=sorted(((scores[i],s)for i,s in enumerate(sentences)),reverse=True)

    summary=[sentence for score,sentence in ranked_sentences[:topn]]
    return " ".join(summary)
nltk.download('punkt_tab')
print(textrank_summarization(text))
```

Results:

```
[57] In 1947, the British Indian Empire was partitioned into two independent dominions, a Hindu-majority dominion of India and a Muslim-majority dominion of Pakistan.
     India, officially the Republic of India,[j][20] is a country in South Asia. [22][23][24] Bounded by the Indian Ocean on the south, the Arabian Sea on the southwest,

and the Bay of Bengal on the southeast, it shares land borders with Pakistan to the west;[k] China, Nepal, and Bhutan to the north; and Bangladesh and Myanmar… nationalist movement emerged in India,

the first in the non-European British Empire and an influence on other nationalist movements.
```

Observation:

Text preprocessing is an important step in NLP that converts raw text into a clean and structured form. Tokenization splits text into words or sentences, while stopword removal reduces unnecessary data. Stemming and lemmatization simplify words, with stemming being faster but less accurate, and lemmatization being more meaningful. Overall, preprocessing improves data quality and helps models perform better.

Post Lab Exercise:

1. Write about "your ownself" (in not more than 500 words→ You know better about you, rather than ChatGPT!!). Generate the summary of your portfolio in
   a. 50% of the size of your portfolio
   b. 25% of the size of your portfolio

   CODE:

```
# Install library
from sumy.parsers.plaintext import PlaintextParser
from sumy.nlp.tokenizers import Tokenizer
from sumy.summarizers.text_rank import TextRankSummarizer
import nltk
nltk.download('punkt_tab')

text = """I am Bhavin Muchhala, a third-year B.Tech student at Marwadi University,pursuing Information
    and Communication Technology.
 I have a strong interest in software development and artificial intelligence.
 I have worked on projects such as an e-commerce website, a bank management system, and a logistics
    project (PackPal) involving route optimization and live tracking.
 I also have experience with microcontrollers and face detection systems using hardware integration"""
```

```
parser = PlaintextParser.from_string(text, Tokenizer("english"))
summarizer = TextRankSummarizer()

# 50% summary (approx 4 sentences)
print("50% Summary:")
for sentence in summarizer(parser.document, 4):
    print(sentence)

# 25% summary (approx 2 sentences)
print("\n25% Summary:")
for sentence in summarizer(parser.document, 2):
    print(sentence)
```

OUTPUT:

```
50% Summary:
I am Bhavin Muchhala, a third-year B.Tech student at Marwadi University,pursuing Information and Communication Technology.
I have a strong interest in software development and artificial intelligence.
I have worked on projects such as an e-commerce website, a bank management system, and a logistics project (PackPal) involving route optimization and live tracking.
I also have experience with microcontrollers and face detection systems using hardware integration

25% Summary:
I am Bhavin Muchhala, a third-year B.Tech student at Marwadi University,pursuing Information and Communication Technology.
I have worked on projects such as an e-commerce website, a bank management system, and a logistics project (PackPal) involving route optimization and live tracking.
```

Comment over the summary obtained. Which sentence you think should be there in your summary, but was not spotted by textrank? Rate the overall summary in the scale of 1-5 (with 1 as least and 5 as highest rating). Paste the code, your portfolio, output and your analysis.

- **Comment on the Summary**
  - The summaries correctly captured:
    - MY **name and academic background**
    - MY **interest in software development and AI**
    - My **major projects**
  - However, some important details were **missed or reduced**, such as:
    - Experience with **microcontrollers**
    - Work on **face detection systems**
    - The **technical depth of PackPal (route optimization + tracking)**

- **Important Missing Sentence**
  This should ideally appear in the summary:
  "I also have experience with microcontrollers and face detection systems using hardware integration."

- **Rating of Summary**
  Rating: 4 / 5
  Reason:
  - Good coverage of **main academic and project details**
  - Slight loss of **technical depth and hardware experience**